

CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



Escuela de Ingeniería

Ingeniería en Ciencias Computacionales

Desarrollo para Plataformas Heterogéneas

Proyecto Final - Documento Técnico

Presenta:

Adam Palomares, 37951

Alonso López, 33744

Karim Ortiz, 32529

Paris Mariscal 33277

Tijuana, B.C., 20 de noviembre de 2025.

Índice

- Página 2. **Planteamiento del problema**
- Página 2-3. **Definición de objetivos**
- Página 3-6. **Arquitectura y justificación**
- Página 7-8. **Diseño general**
- Página 8-11. **Modelo Entidad-Relación**
- Página 11-12. **Descripción del despliegue**
- Página 12. **Enlaces a documentación de los recursos utilizados**

1. Planteamiento del problema

Problema a resolver: La gestión de boletos y registro de eventos todavía se realiza en modalidad digital y presencial, pero en este caso se enfocará en la digital. La oferta de venta digital puede verse afectada por cuestiones de exclusividad. Por ejemplo, una plataforma dedicada puede estar ligada exclusivamente a alguna entidad externa, como algún banco en específico, algún conglomerado de promotores específicos, métodos de pago, patrocinios, entre otros. Estos factores pueden generar problemas como:

- Limitar los métodos de pago.
- Pérdida de asistencias potenciales por pequeño alcance.
- Falta de una experiencia fluida y accesible para organizadores y asistentes.
- Visibilidad de eventos afectada por la asociación a promotores o organizaciones particulares.
- Costos y tarifas adicionales ligadas a servicios externos.

Crear una plataforma hermana, que sirva como una alternativa para la aplicación existente, se traduce en brindar servicio a clientes potenciales. Proporcionando visibilidad a la empresa a distintos públicos objetivos.

2. Definición de objetivos

Diseñar e implementar una plataforma digital de gestión de boletos y registro de eventos que amplíe el alcance de clientes potenciales, mejore la experiencia de organización y reduzca las limitaciones asociadas a servicios externos

Implementar el registro e inicio de sesión de usuarios para permitir la creación de cuentas, autenticación segura y gestión de roles.

Desarrollar la gestión de eventos para facilitar la creación, edición y administración de eventos.

Ofrecer la venta de boletos mediante un sistema en línea con métodos de pago seguros y diversos.

Emitir la generación de boletos electrónicos verificables con códigos QR únicos.

Incorporar notificaciones para enviar confirmaciones, recordatorios y avisos relevantes a los usuarios.

3. Arquitectura y justificación

3.1 Estrategia de desarrollo: SSR

Por qué SSR:

SEO, Descubribilidad y alcance: Las páginas se generan en el servidor y llegan “listas” al navegador, lo que favorece la indexación por buscadores y mejora el tráfico orgánico.

Velocidad percibida: El primer contenido aparece antes (mejor TTFB/First Paint), sobre todo en conexiones móviles o dispositivos modestos. Esto reduce fricción en páginas clave (lista de eventos, detalle, checkout).

Robustez y accesibilidad: Depende menos de que el cliente ejecute mucha lógica; si el dispositivo o el navegador son limitados, la experiencia sigue siendo funcional.

Coherencia de datos y reglas: La composición de página y las validaciones ocurren en un entorno controlado (servidor), lo que disminuye estados incoherentes entre vistas.

Simplicidad operativa: La complejidad de la interfaz se mantiene en límites razonables; es más fácil entregar un MVP estable en un lapso corto.

3.2 Arquitectura: MVC

Por qué MVC:

Separación clara de responsabilidades:

- Modelo: reglas de negocio y datos.

- Vista: presentación y plantillas.
- Controlador: dirección de flujos.

Esto reduce el acoplamiento y facilita cambios sin afectar otras partes.

Mantenibilidad a largo plazo: La estructura por capas y responsabilidades minimiza “deuda estructural”; es más sencillo incorporar nuevas personas al proyecto y localizar errores.

Testabilidad: La lógica de control y de dominio puede aislarse y probarse sin depender de la interfaz; esto mejora la calidad y la confianza en cada entrega.

Previsibilidad y consistencia: Rutas, flujos y convenciones estables hacen que el proyecto crezca de forma ordenada.

3.3 Tecnologías utilizadas:

ASP.NET Core MVC:

Marco de desarrollo web de código abierto, el cual es óptimo para la construcción de aplicaciones y servicios web. Al ser un framework moderno de Microsoft, conviene al ser multiplataforma, sirve en algunos frameworks como el estándar del desarrollo web.

Al utilizar las implementaciones de MVC agrega un código mantenible y fácilmente escalable al separar las responsabilidades. En el caso del proyecto resulta como una solución robusta a una propuesta SSR, adaptándose a un sistema robusto y escalable.

Entity Framework:

ORM moderno de Microsoft que permite interactuar con bases de datos mediante objetos en lugar de consultas SQL directas. Facilita el manejo de datos al ofrecer migraciones, validación y un mapeo claro entre modelos y tablas. En el proyecto aporta una capa de acceso a datos más limpia y mantenible, permitiendo trabajar de forma ágil con entidades y asegurando coherencia en la persistencia.

SQLite:

Motor de base de datos ligero que almacena la información en un solo archivo, sin necesidad de un servidor dedicado. Es ideal para proyectos pequeños o medianos que requieren simplicidad y un despliegue rápido. En el contexto del proyecto funciona como una solución eficiente para entornos de desarrollo o pruebas, manteniendo un rendimiento adecuado y reduciendo la complejidad de configuración.

SMTPClient:

Biblioteca utilizada para el envío de correos electrónicos mediante protocolos SMTP. Permite implementar funcionalidades como recuperación de contraseñas, notificaciones o confirmaciones de registro directamente desde la aplicación. Para el proyecto, proporciona una herramienta confiable para gestionar comunicaciones automatizadas con los usuarios, fortaleciendo la experiencia y soporte dentro del sistema.

ASP.NET Core Identity:

Sistema de autenticación y autorización integrado en ASP.NET Core que gestiona usuarios, roles y credenciales de manera segura. Incluye funcionalidades listas para usar como manejo de contraseñas, verificación en dos pasos, recuperación de cuentas y persistencia de sesiones. En el proyecto añade una capa de seguridad robusta, facilitando la implementación de un sistema de administración de usuarios sin necesidad de construir uno desde cero.

Qrious:

Librería especializada en la generación de códigos QR de forma sencilla y rápida. Permite personalizar tamaño, contenido y estilo del código, facilitando su integración en distintos flujos de la aplicación. Su utilidad en el proyecto radica

en habilitar mecanismos de identificación o confirmación visual para usuarios, como tickets, accesos o validaciones, ofreciendo una solución ligera y adaptable.

Discovery API:

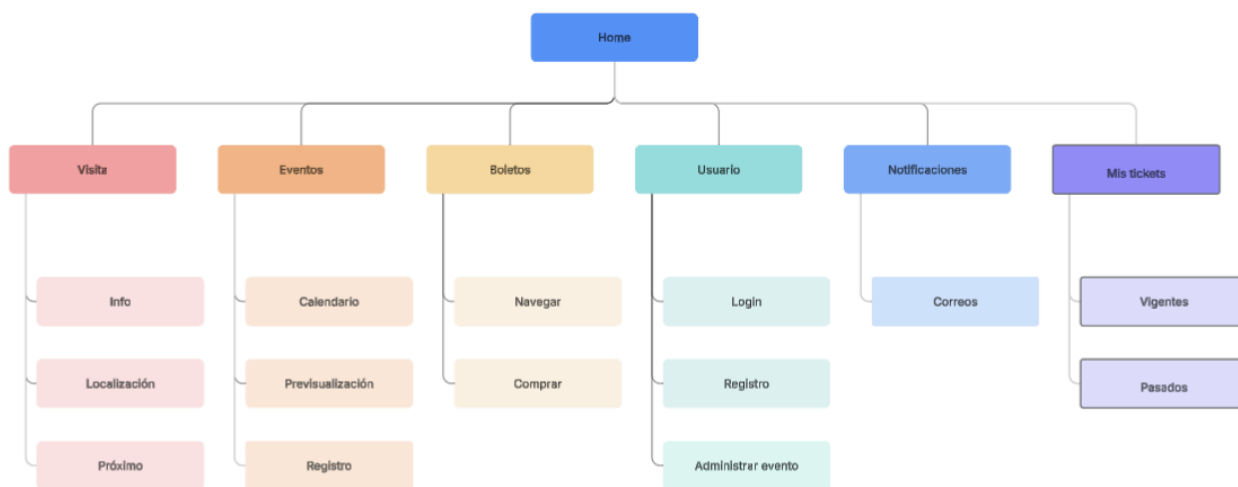
Un servicio REST el cual permite consultar y recuperar información. Ayuda al no necesitar de construirse desde el inicio una base de eventos, pudiendo pasar la lógica de ciertos datos a estas APIs. Su beneficio está al brindar soluciones con acceso a datos en tiempo real sin la necesidad de desarrollar una base de datos tan densa.

Stripe Sandboxes:

Entorno de prueba aislado que simula el funcionamiento de pago sin afectar cuentas ni procesar dinero real. Reproduce todos los flujos de cobro incluyendo autorizaciones, rechazos, reembolsos y autenticación. Beneficia al producto al dar una demo funcional de los procesos de compra de un usuario sin salir del alcance del proyecto.

4. Diseño general

Fig. 1



Explicación del diagrama:

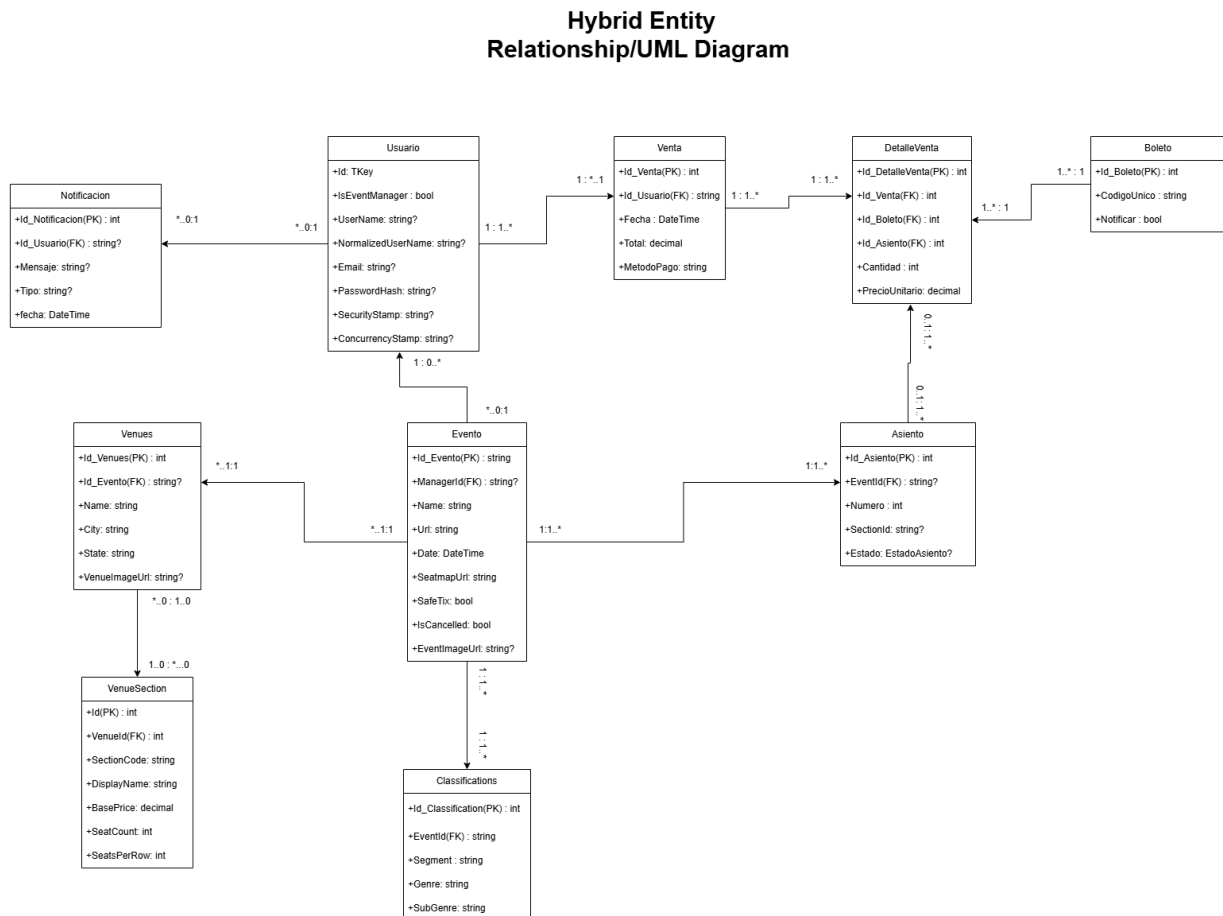
- **Home:** Es la página principal que actúa como un punto de entrada al resto del sistema. Desde aquí, los usuarios podrán acceder a los diferentes módulos de la página.
- **Visita:** Sección informativa de contenido de acceso público. Aquí es donde se encuentra información básica sobre detalles como la ubicación de eventos, descripciones breves y eventos próximos.
- **Eventos:** Sección de la página donde está la gestión de eventos. Permite visualizar un calendario con las fechas, las descripciones detalladas de cada evento y el registro como asistente.
- **Boletos:** Sección para la compra de boletos, los usuarios dentro de esta área también pueden acceder a su historial de compras.
- **Usuario:** Área dedicada para la gestión de cuentas. Incluye el registro de usuarios nuevos y la administración de perfiles de la página.
- **Notificaciones:** Proporciona detalles sobre los correos de los boletos si fue seleccionada la opción al final de la compra.

- **Mis Tickets:** Muestra los tickets de un usuario en dos secciones distintas, vigentes o activos, que son los boletos con eventos próximos y pasados, que son los eventos que no están actualmente activos por pasarse de la fecha de inicio.

5. Diagrama de entidades del modelo de negocio

a. Modelo híbrido entre ER y UML sobre las entidades del programa.

Fig. 2

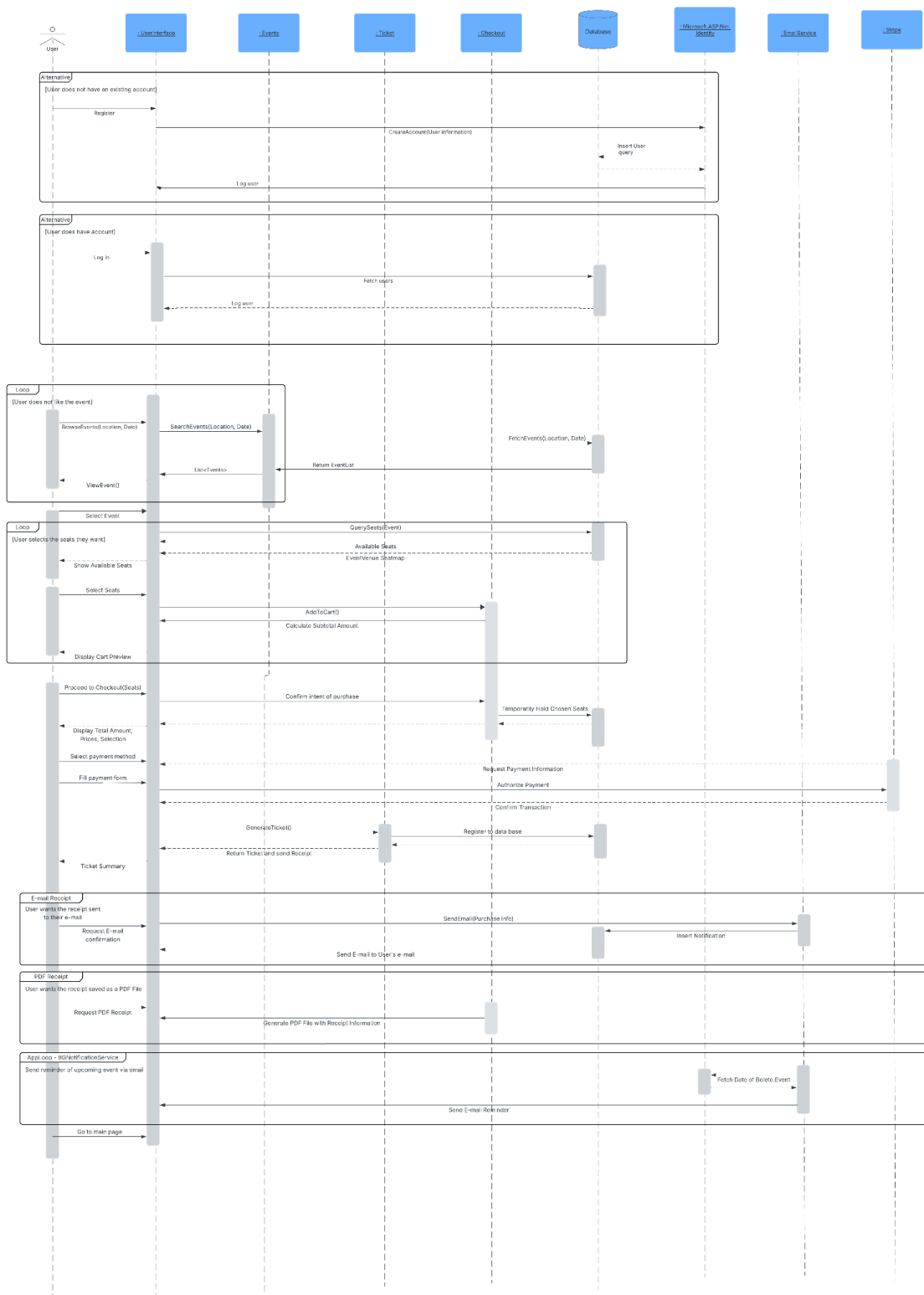


Nota: En la imagen se aprecian los componentes y conexiones de la base de datos. La entidad central es el evento, ya que concentra la mayor cantidad de relaciones. Esto refleja que es el actor principal que utiliza la mayor parte de la información contenida en el sistema. Un ejemplo de lo que vería el consumidor

sería el siguiente: el usuario accede a la aplicación y realiza el login; posteriormente, desea realizar la compra de un boleto en un asiento disponible, lo que genera una venta junto con su detalle de venta y, de forma simultánea, se genera el boleto. Finalmente, el usuario recibe notificaciones con recordatorios sobre la fecha y detalles del evento adquirido.

b. Operaciones soportadas por cada modelo

Fig. 3



Nota: La imagen superior representa el flujo real de lo que sucede en el programa. Anteriormente, en la Figura 2, se explicó un caso general de lo que un usuario haría en BigFourApp. Esta sección sólo detalla las operaciones involucradas en la “caja negra” y la secuencia esperada.

6. Descripción del despliegue

Para el despliegue de la aplicación se utilizó Microsoft Azure como proveedor de cómputo en la nube. El proyecto se implementó siguiendo el modelo de servicio PaaS (Platform as a Service), el cual permite ejecutar aplicaciones en una máquina virtual sin gestionar su infraestructura de manera directa, un modelo ideal para la publicación de aplicaciones web.

El servicio principal empleado para la ejecución de la aplicación fue Azure App Service, una plataforma diseñada para alojar aplicaciones web de manera escalable, segura y con alta disponibilidad. Este servicio se encargó de la asignación de recursos, balanceo de carga, entorno de ejecución y la supervisión del estado de la aplicación, lo que permitió el despliegue sin necesidad de administrar un servidor manualmente.

- Máquina Virtual Linux Ubuntu 24.04:
 - Standard D2s v3
 - vCPU: 2
 - RAM: 8 GiB
 - Región: West US
- Microsoft Azure App Service:
 - Dotnet Core 9.0
 - Región: Canadá Central

Para el manejo de archivos subidos por los gestores de eventos (Imágenes de evento y mapas de asiento) la aplicación utilizó Azure Blob Storage como solución de almacenamiento, un servicio que corresponde también al modelo

PaaS, orientado al almacenamiento de objetos de forma escalable y persistente. Blob Storage proporcionó URLs para el acceso a los archivos, asegurando que se mantuvieran disponibles incluso después de reinicios, actualizaciones o nuevos despliegues de la aplicación; al igual que se ofrecen funcionalidades de respaldo, que mejoran la experiencia de gestión de archivos.

En conjunto, Azure App Service y Azure Blob Storage permitieron implementar una arquitectura en la nube administrada, estable y adaptable a las necesidades técnicas de la aplicación.

7. Enlaces

- Librería Qrious. <https://github.com/neocotic/qrious>
- TicketmasterDiscoveryAPI.
<https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/>
- Entity Framework. <https://learn.microsoft.com/en-us/ef/>
- Azure App Service.
<https://azure.microsoft.com/es-es/products/app-service>
- Azure Blob Storage.
<https://azure.microsoft.com/es-es/products/storage/blobs>